

On-Premises Local AI Financial Audit & Business Intelligence Assistant

A Multi-Modal, Privacy-Preserving Agentic Decision System

Course: Engineering Design and Innovation (EDI) - Semester IV

Architecture: LangGraph Orchestrated State Machine / RAG / Computer Vision

AI Models: Local Llama 3 (Ollama) & Microsoft Florence-2 Base VLM

Database: Relational PostgreSQL in Docker & Persistent ChromaDB Vector Store

1. Executive Summary & Project Goal

Modern enterprises handle massive volumes of financial transactions, employee expense sheets, and invoices daily. Auditing these records to detect discrepancies, fraud, or violations of corporate compliance policies is incredibly complex. Furthermore, sending proprietary ledgers and scanned visual receipts to external cloud-hosted LLM services (like OpenAI or Anthropic) presents substantial data privacy risks and security liabilities.

The Solution: This project implements an entirely **on-premises, local-first AI assistant** designed to perform automated, multi-modal financial audits and high-fidelity business intelligence analysis. The system runs fully offline on local consumer-grade GPU/CPU hardware, keeping all enterprise data secure within the organizational firewall.

By orchestrating multiple specialized AI modules, the system accomplishes three core objectives:

- **Structured Data Querying:** Translates colloquial English questions into complex PostgreSQL queries using a local text-to-SQL pipeline.
- **Dynamic Business Intelligence:** Parses database rows and generates custom visual charts (pie, bar) programmatically using Matplotlib.
- **Multi-Modal Compliance Auditing:** Performs a three-way verification matching the **Database Ledger entry**, the scanned **physical receipt's visual OCR**, and **corporate guidelines** retrieved via a Vector Database.

2. System Architecture & High-Level Workflow

The application is structured as a multi-layered agentic system coordinated by a LangGraph workflow. The modular design separates parsing, database access, document search, and orchestration into independent nodes.

The workflow transitions through several core phases:

- 1. Input Query & Planning:** The user submits an arbitrary text command via a REST API. The *Planner Node* analyzes the prompt and classifies the intent using a strict Pydantic JSON schema.
- 2. Decision-Based Routing:** Based on the intent, the LangGraph router directs execution into one of three isolated lanes:

Lane	Execution Workflow
Visualization Lane	Fetches SQL data → structures values using LLM → renders and saves a high-quality chart using Matplotlib.
Analysis Lane	Fetches relational rows → feeds them to the LLM → synthesizes a short, precise corporate performance summary.
Audit Lane	Extracts transaction ID → queries PostgreSQL ledger → scans visual receipt using Florence-2 VLM → queries ChromaDB for relevant policies → synthesizes final compliance report.

3. Detailed File-by-File Breakdown

The codebase is modular, keeping data generation, ML processing, relational database management, and HTTP APIs fully separated. Below is a comprehensive list and technical breakdown of each file in the workspace:

File Name	Detailed Purpose & Features
<code>docker-compose.yml</code>	Type: Configuration File Technologies: Docker, PostgreSQL 15 Alpine Detailed Description: Defines the local container services. It spins up a PostgreSQL relational database on port 5432, pre-configured with the user <i>audit_admin</i>, database name <i>corporate_ledger</i>, and a secure password. A Docker volume is mounted to make the data persistent.
<code>setup_db.py</code>	Type: Database Initiator Technologies: SQLite, Pandas, Random Detailed Description: Generates a baseline local SQLite database file named <i>corporate_ledger.db</i> with an <i>expenses</i> table. It generates 2,500 random corporate transaction rows spanning multiple categories (Travel, Hardware, software, Food & Bev, Consulting, Shipping) to serve as sample data.
<code>data_pipeline.py</code>	Type: Production Data Generator Technologies: PostgreSQL, SQLAlchemy, Pandas, Faker Detailed Description: Creates a more realistic dataset consisting of 1,500 enterprise rows. Unlike <code>setup_db.py</code>, it inserts this data directly into the running PostgreSQL Docker container. It defines category-specific price bounds (e.g. software/hardware ranges from \$1,000 to \$25,000 while travel ranges from \$50 to \$3,000) for realistic business metrics.
<code>migrate_to_postgres.py</code>	Type: Database Migration Script Technologies: SQLAlchemy, Pandas, SQLite, PostgreSQL Detailed Description: Performs a database migration. It queries the local SQLite file's system catalogs, extracts all database tables, and copies their entire schema and records into the running PostgreSQL container, safely replacing outdated rows.
<code>generate_docs.py</code>	Type: Mock Unstructured Data Generator Technologies: ReportLab, Pillow (PIL) Detailed Description: Programmatically builds PDF policies (e.g., travel policies with VP spending thresholds and approved hardware vendors) and physical receipt images (PNG files) with embedded transaction details. It injects intentional anomalies (e.g. TXN-1004 has an image amount of \$990 but a database amount of \$890, and TXN-1003 exceeds the VP travel budget limit of \$1000) for testing AI accuracy.
<code>generate_receipts.py</code>	Type: Dynamic Bulk Receipt Generator Technologies: PostgreSQL, SQLAlchemy, Pandas, Pillow (PIL), Random Detailed Description: Fetches the live transaction database from PostgreSQL, samples 50 rows, and creates highly realistic scanned receipts for them. It injects random anomalies: 15% have ledger mismatches (fraud), 15% are missing completely, and 70% match perfectly.

<code>ingest_pdfs.py</code>	Type: Knowledge Ingestion Pipeline Technologies: LangChain, PyPDFDirectoryLoader, ChromaDB, HuggingFaceEmbeddings Detailed Description: Processes raw unstructured policy PDFs in <i>audit_files/policies</i>. It loads pages, divides them into small 500-character blocks with 50-character overlaps, converts them into high-dimensional vectors via the <i>all-MiniLM-L6-v2</i> model, and saves the records into a local vector database directory called <i>chroma_db/</i>.
-----------------------------	---

File Name	Detailed Purpose & Features
chat_with_pdfs.py	Type: Conversational Policy Engine Technologies: ChromaDB, LangChain, ChatOllama (Llama 3) Detailed Description: Features a command-line RAG application that queries local corporate compliance guidelines. It retrieves the 3 most relevant context blocks from ChromaDB based on the user's prompt and sends them to Llama 3 for a precise, context-bounded compliance answer. It also houses `query_policy` for programmatic LangGraph lookups.
sql_tools.py	Type: Natural Language Database Tool Technologies: LangChain, SQLDatabase, ChatOllama (Llama 3) Detailed Description: Implements a robust text-to-SQL utility. When fed a sentence, it dynamically fetches the PostgreSQL catalog schema, generates correct SQL, executes the query via LangChain's SQL runner, and pipes the outcome and original prompt to Llama 3 to formulate a natural response.
planner_node.py	Type: Intent Classifier & Step Planner Technologies: Pydantic, LangChain, ChatOllama (Llama 3) Detailed Description: Standardizes inputs entering the system. Using strict Pydantic schemas, it asks Llama 3 to output JSON containing the user's underlying category (intent) and a list of tool steps (e.g. <i>[fetch_sql_data, generate_chart]</i>) to follow.
receipt_parser.py	Type: Vision OCR & Formatting Node Technologies: PyTorch, Transformers, Florence-2 Base, ChatOllama Detailed Description: Orchestrates visual document parsing. It passes visual invoice images to Microsoft's Florence-2 model to extract printed characters using the model's native <code><OCR></code> task. It then uses Llama 3 to parse the unstructured text block into a rigid JSON structure (<code>{vendor, txn_id, amount}</code>). Includes a patch to run locally without requiring flash attention dependencies.
master_agent.py	Type: Workflow Orchestration Core Technologies: LangGraph, Matplotlib, ChatOllama (Llama 3), Regex Detailed Description: The central state machine. It establishes the LangGraph nodes, routing edges, and shared memory (BIState). It coordinates executing the Visualization Lane (programmatic Matplotlib charts), Analysis Lane (conversational stats analysis), or the Audit Lane (a multi-modal audit verifying database logs vs receipt OCR vs company policy RAG contexts).
api.py	Type: REST API Daemon Gateway Technologies: FastAPI, Uvicorn, CORS, StaticFiles Detailed Description: Serves the backend AI to user interfaces. It runs on port 8123 and features a single POST endpoint <code>/ask</code>. It accepts text queries, invokes the compiled LangGraph agent, returns the synthesized textual report, and maps static file paths to display dynamically generated charts.
test_llm.py	Type: Diagnostics Script Technologies: LangChain, ChatOllama (Llama 3) Detailed Description: A simple diagnostics tool to verify that the local connection to your Ollama model is operating properly. It outputs standard test code while timing local model generation speeds.

4. The 3-Way Auditing Match Logic

The crown jewel of the system is the **Multi-Modal Audit Lane** in `master_agent.py`. It automates checks that typically consume hours of manual audit time. Here is exactly how the 3-way matching logic operates:

Step 1: Database Querying

Using the identified transaction ID (e.g. *TXN-1004*), the agent queries the PostgreSQL database using SQL tools to pull the formal amount and vendor recorded on the ledger: `db_data = sql_response['answer']`.

Step 2: Scanned Receipt Analysis (Vision VLM)

The agent checks the physical server for a scanned file matching the pattern `receipt_TXN-1004.png`. It sends this picture through the **Florence-2 vision model**. The vision model performs a dense Optical Character Recognition (OCR) scan to extract all visible text. That text is immediately passed to a local Llama 3 formatter that extracts the vendor and actual visual invoice total: `receipt_data = extract_receipt_data(image_path)`.

Step 3: Corporate Policy Verification (RAG)

The agent automatically performs a vector lookup query inside ChromaDB: *'What is the policy for receipt discrepancies, matching amounts, and manual review thresholds?'* and pulls the exact text matching those policies.

Step 4: AI Audit and Compliance Verdict

The agent feeds the outputs of all three channels into a final Llama 3 prompt, acting as a strict audit supervisor. It evaluates:

1. Did the database ledger amount match the visual receipt total exactly?
2. Does the expense violate vendor limitations (e.g., procurement of non-approved brands)?
3. Does the transaction size exceed specific executive limits requiring documented VP approval?

Finally, the AI outputs a concise corporate verdict: **APPROVED** or **FLAGGED FOR REVIEW** along with a clear summary.

5. Execution & Deployment Guide

To run the complete workspace locally in a unified flow, follow these terminal steps:

```
# 1. Start the local database container
docker-compose up -d

# 2. Build the fake databases, documents, and policies
python setup_db.py
python generate_docs.py
python ingest_pdfs.py

# 3. Migrate database tables and load PostgreSQL
python migrate_to_postgres.py
python data_pipeline.py
python generate_receipts.py
```

```
# 4. Boot up the local web service gateway  
python api.py
```